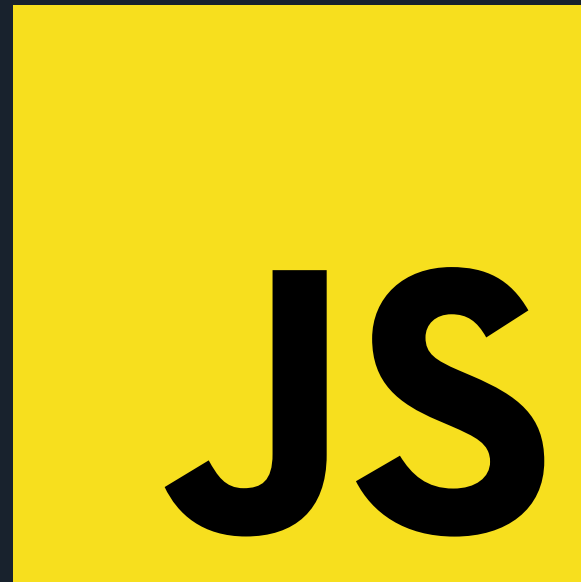


JAVASCRIPT



JAVASCRIPT

Introduction

Le JavaScript est un langage de programmation, parmi les plus utilisés au monde

- Le JavaScript est créé en 1995
- La syntaxe est standardisée sous le nom d'*ECMAScript*
- Depuis 2015, une nouvelle version sort chaque année
- Le web utilise ce langage pour scripter ses pages, tous les navigateurs savent interpréter du JavaScript

Il existe un langage de programmation nommé Java, qui n'a absolument rien à voir avec JavaScript.
Il ne faut pas les confondre

No notes on this slide.

JAVASCRIPT

Ressources

De nombreux tutoriels et documentations sont disponibles sur presque tous les supports (livres, sites internet, vidéos, etc.)

- W3Schools : <https://www.w3schools.com/js/>
Références et tutoriels facile d'accès. Excellent pour débiter
- MDN : <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
Documentation officielle de Mozilla. Référence la plus complète et la plus précise disponible

N'hésitez pas à chercher sur internet dès que vous avez une questions.

Savoir utiliser une documentation doit faire partie de vos compétences

No notes on this slide.

JAVASCRIPT

Comment ca marche ?

Le JavaScript (plus précisément ECMAScript) est une syntaxe qui décrit des opérations à effectuer. Comme pour la plupart des langages de programmation, l'exécution se fait ligne par ligne.

L'exécution est faite par un programme spécifique, un *moteur JavaScript*, intégré au navigateur web.

Il existe d'autres moteurs JavaScript qui ne sont pas intégrés à un navigateur, comme *Node.js* que nous verrons plus tard.

JAVASCRIPT

Dans une page web

En HTML, le javascript s'écrit toujours via une balise `<script>`.

Il peut s'écrire directement dans la balise :

```
<!DOCTYPE html>
<html>
  <head>
    <title>JavaScript</title>
  </head>
  <body>
    <script>
      console.log('Hello World!');
    </script>
  </body>
</html>
```

JAVASCRIPT

Dans une page web

En HTML, le javascript s'écrit toujours via une balise `<script>`.

Ou référencer le contenu d'un fichier externe :

```
<!DOCTYPE html>
<html>
  <head>
    <title>JavaScript</title>
  </head>
  <body>
    <script src="script.js"></script>
  </body>
</html>
```

Ces deux codes sont équivalents si *script.js* contient `console.log('Hello World!');`

JAVASCRIPT

L'exécution

Le JavaScript s'exécute dans l'ordre de lecture de la page par le navigateur. Cela implique qu'un script ne peut accéder qu'aux éléments déjà analysés.

Par défaut, l'exécution du JavaScript est bloquante pour le chargement de la page. On placera généralement la balise `<script>` à la fin de la balise `<body>` pour ne pas ralentir le chargement.

```
<!DOCTYPE html>
<html>
  <head>
    <title>JavaScript</title>
    <script>...</script> -> Exécuté avant le chargement du body, il ne pourra pas y accéder
  </head>
  <body>
    <script>...</script> -> Exécuté après le chargement du body et avant celui du h1
    <h1> Titre </h1>
    <script>...</script> -> Exécuté une fois tous les éléments chargés
  </body>
</html>
```

Cet ordre d'exécution est également valable pour les balises de script qui incluent un fichier externe. Cela ne fait pas de différence.

Il existe en JavaScript des événements qui permettent d'exécuter du code à un moment donné (lorsque la page s'est chargée, lors du clic sur un bouton, quand une ressource est disponible, etc). Ces derniers permettent aussi de contrôler l'ordre d'exécution. Nous en verrons certains plus loin dans le cours.

JAVASCRIPT

Les variables

JavaScript utilise un *typage implicite*.

Comme en Python, on ne déclare pas le type des variables. Les variables sont déclarées avec `let`, ou avec `const` si leur valeur ne change pas.

```
let maVariable = 3;    // Déclare une variable, sa valeur pourra changer
const maVariable = 3;  // Déclare une constante, changer sa valeur provoquera une erreur
```

Selon la norme ECMAScript, toutes les instructions en JavaScript doivent se terminer par un point-virgule, mais l'interpréteur acceptera si vous ne les mettez pas. Prenez l'habitude de respecter le standard et de toujours les utiliser.

JAVASCRIPT

Les variables

Il est possible de déclarer des variable avec `var` ou sans utiliser de préfixe. Cela déclare une variable globale au contexte courant.

On ne l'utilise plus en pratique, cela créé des confusions.

```
// Ces deux lignes font la même chose. maVariable sera globale à la fonction courante
maVariable = 3;
var maVariable = 3;
// Ces syntaxes sont désuètes, à ne pas utiliser
```

Il est aussi recommandé de terminer toutes les lignes par un point-virgule car cela permet d'éviter des erreurs subtiles de syntaxe. Par exemple, le code suivant :

```
const a = 1 + 2 ("a" + "B").toLowerCase()
```

 Retournera l'erreur : *Uncaught TypeError: 2 is not a function* Car JavaScript interprète un retour à la ligne comme une fin d'instruction **uniquement** quand cela est la seule solution syntaxique. Or la ligne

```
const a = 1 + 2("a" + "B").toLowerCase()
```

 est syntaxiquement valide, et donc interprétée comme telle. Mais elle retourne une erreur lors de l'exécution car "2" suivit de parenthèses ne veut rien dire. Avec des points-virgules cette erreur ne serait pas arrivée.

JAVASCRIPT

Les types

Bien que le typage soit implicite, toutes les variables en JavaScript possèdent un type.

Il existe 7 types de base.

```
const maVariable = "JavaScript";    // Chaîne de caractères
const maVariable = 3.14;            // Nombre réel
const maVariable = true;            // Booléen
const maVariable = null;            // null, indique l'absence d'un objet
const maVariable = undefined;       // undefined, indique "pas de valeur", "non déclaré"
const maVariable = BigInt(3);       // BigInt, entier sans limites
const maVariable = Symbol("foo");   // Symbol, pas utile dans ce cours
```

L'instruction `typeof` retourne le type de la variable sous forme de chaîne de caractères

```
const maVariable = false;
const typeDeMaVariable = typeof maVariable; // typeDeMaVariable vaut "boolean"
```

Il existe également d'autres types non basiques, qui sont des structures stockant des types basiques (tableaux, objets, etc).

JavaScript est un langage à **typage dynamique implicite**
 Implicite signifie que le langage infère le type selon le contexte, on n'indique jamais explicitement le type d'une variable
 Dynamique signifie que la vérification des types est faite lors de l'exécution. Par opposition à des langages tels que le C/C++ où la vérification des types est faite lors de la compilation.
 Même si on ne déclare pas explicitement le type d'une variable, JavaScript assigne en interne un type à chaque variable.

JAVASCRIPT

La console

Pour afficher un texte dans la console :

```
console.log("Bonjour"); // Affichera "Bonjour" dans la console  
console.warn("Attention"); // Affichera un message d'avertissement  
console.error("Erreur"); // Affichera un message d'erreur
```

La fonction print() existe en JavaScript, mais il s'agit de la commande pour imprimer la page web !

La console des navigateurs web permet d'afficher plus que du texte. Il est possible d'afficher et d'explorer des variables, des objets, etc. Utilisez-la pour debugger votre code.

A part les types de bases, tous les autres sont des "objects". Cela veut dire que tous les types de JavaScript suivent une logique similaire et peuvent facilement être affichés dans la console. La console des navigateurs, quand un objet y est affiché, ajoute même une interface pour naviguer dans les propriétés de l'objet. Cela est extrêmement pratique pour debugger.

JAVASCRIPT

Les opérations booléennes

Les comparaisons se font de manière classique :

```
console.log(3 < 2);      // false
console.log(3 >= 2);     // true
console.log(3 == 2);     // false
console.log(3 != 2);     // false
```

Les opérateurs === et !== permettent de tester la valeur et le type :

```
console.log(2 == "2");   // true      car les valeurs sont identiques, peu importe le type
console.log(2 === "2");  // false     car les types sont différents
console.log(2 != "2");   // false
console.log(2 !== "2");  // true
```

Les opérateurs && , || et ! fonctionnent respectivement comme and, or et not en Python

```
console.log( !(2 != "2" || 3 < 2) && 0 < 4); // true
```

JAVASCRIPT

Les opérations mathématiques

En JavaScript on retrouve les opérations de base : addition +, soustraction -, multiplication *, division /, modulo % (reste d'une division entière).

Il existe également les opérateurs d'assignement : +=, -=, *=, /=

$x += y$ est équivalent à $x = x + y$, idem pour les autres opérateurs

Il existe aussi les opérateurs d'incrémentation ++ et de décrémentation --

$x++$ est équivalent à $x = x + 1$ et $x--$ est équivalent à $x = x - 1$

L'opérateur + sert de concaténation pour les chaînes de caractères. "Hello " + "World" donne "Hello World"

Pour tous les détails, voir https://www.w3schools.com/js/js_operators.asp

JAVASCRIPT

Les fonctions mathématiques

Toutes les fonctions de math avancées se font grâce à l'objet Math

- `Math.abs(x)` : valeur absolue
- `Math.log(x)`, `Math.exp(x)` : logarithme et exponentielle
- `Math.min(x, y, z, ...)`, `Math.max(x, y, z, ...)` : minimum et maximum de plusieurs valeurs
- `Math.random()` : nombre aléatoire dans l'intervalle $[0, 1[$
- `Math.sqrt(x)`, `Math.pow(x, y)` : racine carrée et puissance
- `Math.round(x)`, `Math.floor(x)`, `Math.ceil(x)` : Arrondi resp. au plus proche, inférieur, supérieur
- `Math.PI` : π

Référence complète : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Math

JAVASCRIPT

Coercition

Les opérateurs mathématiques peuvent être utilisés entre tout types, mais attention aux comportements surprenants !

L'addition est particulière.

<code>console.log(3 + 5);</code>	<code>// 8</code>	Addition standard
<code>console.log("3" + "5");</code>	<code>// "35"</code>	Concaténation de texte
<code>console.log(3 + "5");</code>	<code>// "35"</code>	Dès qu'un opérateur est un texte, c'est une concaténat
<code>console.log(3 - 5);</code>	<code>// -2</code>	Soustraction standard
<code>console.log("3" - "5");</code>	<code>// -2</code>	La soustraction converti automatique tout texte en non
<code>console.log("3" - "A");</code>	<code>// NaN</code>	Si un texte n'est pas convertible, retourne la valeur
<code>console.log("3" * "5");</code>	<code>// 15</code>	Idem pour tous les autres opérateurs
<code>console.log("3" * true);</code>	<code>// 3</code>	true vaut 1, false vaut 0
<code>console.log("3" - null);</code>	<code>// 3</code>	null est converti en la valeur 0
<code>console.log("3" - []);</code>	<code>// 3</code>	Les tableau vides valent 0
<code>console.log("3" - undefined);</code>	<code>// NaN</code>	Une opération avec "undefined" provoquera toujours un

Il existe beaucoup d'autres cas. On appelle cette "conversion automatique" entre types la *coercition*.

Retenez : Éviter au maximum les opérations entres types différents !

JAVASCRIPT

Cas particuliers

Pas convaincu de ne pas mélanger les types ?

```
console.log(('b' + 'a' + + 'lancer' + 'a').toLowerCase()); // Qu'affiche ceci ?
```

```
console.log('b' + 'a' + (+ 'lancer') + 'a'); // 'a'++ n'est pas valide
```

```
console.log('b' + 'a' + NaN + 'a'); // 'lancer' est converti en nombre, invalide donc NaN
```

```
console.log('baNaNNa'); // NaN est converti en texte via l'addition
```

```
console.log('baNaNNa'.toLowerCase()); // Met le texte en minuscules
```

```
console.log(('b' + 'a' + + 'lancer' + 'a').toLowerCase()); // Affichera "banana"
```

On entend beaucoup de critiques de JavaScript à cause de ces comportements bizarres. Pour nuancer un peu, il s'agit la majorité du temps de codes qui ne s'écrivent pas en pratique.

N'oubliez pas : gardez toujours votre code propre et clair. En faisant ça vous évitez ce genre de problèmes.

Liste d'opérations WTF en JavaScript: <https://github.com/denysdovhan/wtfjs>

JAVASCRIPT

Les conditions

Les conditions s'écrivent de la manière suivante :

```
if(a < b){  
    console.log("Do A");  
}else if(a < c){  
    console.log("Do B");  
}else{  
    console.log("Do C");  
}
```

Comme en Python, les blocs `else if` et `else` sont facultatifs

Il est aussi possible de faire des *ternaires* avec la syntaxe `condition ? si_vrai : si_faux`

```
const a = 5;  
const value = a > 3 ? "Plus grand" : "Plus petit ou égal";  
console.log(value); // "Plus grand"
```

Détail concernant les conditions: JavaScript (comme Python, C/C++ et bien d'autres) évalue les conditions en court-circuit. Cela veut dire que JavaScript évalue les conditions élément par élément, et continue l'exécution dès qu'une décision peut être prise, même si toute l'expression n'a pas été évaluée. Exemple :

```
const a = [];  
  
// Cette ligne est valide, car comme "a.length <= 0" est vrai, la condition s'exécute sans évaluer la  
if(a.length <= 0 || a[0][0] == 2){ console.log("Valide"); }  
  
// Cette ligne provoque une erreur, car "a[0]" est undefined et donc "a[0][0]" est invalide  
if(a[0][0] == 2 || a.length <= 0){ console.log("Erreur"); }
```

JAVASCRIPT

Les boucles — la boucle while

Pour répéter des instructions plusieurs fois, il existe les boucles

```
let text = "";
while(text != "Bonjour"){
  text = prompt("Dites 'Bonjour' !"); // prompt() Demande à l'utilisateur d'entrer un texte
}
console.log("Vous avez dit bonjour !");
```

La boucle `while` répète son contenu tant que la condition est vraie. La boucle `while` est utile quand *on ne connaît pas le nombre de répétitions à faire*. Attention à *toujours* vous assurer que la boucle se terminera, sinon c'est une boucle infinie

L'évaluation de la condition est faite au début de chaque itération

Les boucles infinie sont particulièrement dangereuses car elles bloquent complètement la page, et il n'y a pas d'autres choix que de recharger la page.

S'il n'est pas évident que la boucle s'arrêtera quoi qu'il arrive, pensez à mettre une condition d'arrêt, ou utiliser une boucle *for* à la place.

JAVASCRIPT

Les boucles — la boucle for

La boucle `for` est une autre manière de répéter des instructions

```
for(let i=0; i<10; i++){  
  console.log(i); // 0 1 2 3 4 5 6 7 8 9  
}
```

La boucle `for(initialisateur ; condition ; iteration )` :

- Exécute `initialisateur` avant d'entrer dans la boucle
- Exécute `iteration` après chaque tour de boucle
- S'exécute tant que `condition` est vraie

Ce type de boucle est utile quand *on connaît le nombre de répétitions à faire*.

Elle permet d'éviter des erreurs qui créent des bouclent infinies, car il est plus simple de voir que la boucle s'arrêtera dans tous les cas.

Toute boucle *for* peut être écrite sous forme de boucle *while* et inversement. Choisir l'une ou l'autre dépend du contexte. On préférera le type de boucle qui permet d'exprimer le plus simplement la logique du code.

JAVASCRIPT

Les boucles — break & continue

Dans une boucle, l'instruction `break` sort immédiatement de la boucle, sans condition.

```
const a = [/*...*/];
for(let i=0; i<a.length; i++){
  if (a[i] == 0) {           // Si a[i] vaut zéro, sort de la boucle
    break;
  }
  console.log(`${1/a[i]}`); // Affichera l'inverse des éléments du tableau, jusqu'au premier 0
}
```

Dans une boucle, l'instruction `continue` passe immédiatement à l'itération suivante.

```
const a = [/*...*/];
for(let i=0; i<a.length; i++){
  if (a[i] == 0) {           // Si a[i] vaut zéro, passe à l'itération suivante
    continue;
  }
  console.log(`${1/a[i]}`); // Affichera l'inverse des éléments du tableau, sauf ceux valant 0
}
```

Les instructions *break* et *continue* affectent toujours la boucle courante (la boucle la plus intérieure dans le cas de boucle imbriquées).

Toutes les boucles utilisant *break* et *continue* peuvent se réécrire sans ces mots clés.

Comme pour les choix entre *while* et *for*, il faut préférer l'option qui produit le code le plus facilement compréhensible.

JAVASCRIPT

Les tableaux

Les tableaux sont l'équivalent des listes en Python

```
const a = []; // Tableau vide
const b = [1, 2, 3]; // Tableau de nombres
const c = [1, "a", null, 12.4, [1,2] ]; // Les tableaux peuvent contenir un mélange de types
console.log(c[1]); // Accès aux éléments : affiche "a".
c[58] = true; // On peut assigner n'importe quel élément d'un tableau
// Tous les éléments non assignés valent 'undefined'

b.push(5); // "push" ajoute un élément, b vaut alors [ 1, 2, 3, 5 ]
b.pop(); // "pop" supprime et renvoie le dernier élément
// b contient alors [1, 2, 3]

console.log(b.length); // Affiche 3. "length" retourne la taille du tableau

const d = [[1,2,3], [4,5,6], [7,8,9]]; // Il est possible d'avoir des tableaux de tableaux
console.log(d[1][2]); // Affiche 6
```

La numérotation des tableaux commence toujours à 0. `tab[3]` affichera donc le 4eme élément de `tab`.

Techniquement, en JavaScript, les tableaux sont des objets. `typeof []` retourne "object"

En incluant des tableaux dans des tableaux, il est possible de créer des tableaux avec n'importe quelle dimension. Par exemple

```
const m = [
  [ 1,2], [ 3, 4] ],
  [ 5,6], [ 7, 8] ],
  [ 9,10], [11,12] ]
];
```

est un tableau à 3 dimensions auquel on accédera, par exemple, par `m[2][1][0]`.

JAVASCRIPT

Les fonctions des tableaux

- Taille d'un tableau : `array.length`
- Ajouter un élément à la fin: `array.push(2)`
- Retirer le dernier élément : `array.pop()`. Renvoi l'élément retiré
- Trier le tableau : `array.sort()` Trie le tableau et le renvoie
- Inverser l'ordre des éléments : `array.reverse()`. Inverse l'ordre des éléments et renvoie le tableau
- Appartenance : `array.includes('value')`. Retourne 'true' si le tableau contient 'value', 'false' sinon

Et bien d'autres : https://www.w3schools.com/js/js_array_methods.asp

En incluant des tableaux dans des tableaux, il est possible de créer des tableaux avec n'importe quelle dimension. Par exemple

```
const m = [
  [ 1,2], [3, 4] ],
  [ 5,6], [7, 8] ],
  [ 9,10], [11,12] ]
];
```

est un tableau à 3 dimensions auquel on accédera, par exemple, par `m[2][1][0]`.

JAVASCRIPT

Les objets

Un objet en JavaScript est équivalent à un dictionnaire en Python

```
const object = {  
  key1 : 3,  
  key2 : "Value",  
  key3 : {  
    subkey1 : true,  
    subkey2 : 5  
  },  
  key4 : [4,5,6]  
};
```

Un objet est un ensemble *clés* possédant chacune une *valeur*

On accède à une valeur à l'aide de la clé, soit à la manière d'un tableau soit avec un point :

```
console.log(object['key2']); // "Value"  
console.log(object.key2); // "Value"
```

JAVASCRIPT

Les objets

Il est possible de créer une clé directement en l'assignant

```
object['newkey'] = "newValue";  
object.otherNewKey = "OtherNewValue";
```

Il est possible de supprimer une clé avec l'instruction delete

```
delete object['newkey'];  
delete object.otherNewKey;
```

No notes on this slide.

JAVASCRIPT

Traverser des tableaux et des objets

Il existe la boucle `for(... of ...){}` pour itérer dans des tableaux :

```
const table = ["a", "c", "e", "g"];
for(const value of table){
  console.log(value) // a c e g
}
```

Il existe la boucle `for(... in ...){}` pour itérer dans des objets :

```
const obj = {
  k1 : "v1",
  k2 : 3,
  k3 : true
};
for(const key in obj){
  console.log(key + " : " + obj[key]); // k1 : v1    k2 : 3    k3 : true
}
```

A noter que dans ces deux types de boucles, l'itérateur peut être déclaré avec ``const`` (``const value`` et ``const key``). Cela est possible car ces boucles créent une nouvelle variable à chaque itération. Le ``const`` assure ici que la valeur n'est pas modifiée durant une itération.

Modifier les itérateurs "value" ou "key" (s'ils ne sont pas const) ne changera pas l'objet itéré.

JAVASCRIPT

L'immuabilité

En JavaScript, il existe une différence fondamentale entre les types de base et les objets.

Qu'affiche ce code ?

```
let a = 5;  
let b = a;  
  
b = 10;  
  
console.log(a);  
console.log(b);
```

Ce code affichera :

5

10

Rien de très surprenant

Voici un exemple simple, rien de surprenant, mais il est intéressant d'examiner en détail ce que JavaScript fait lors de l'exécution de ces lignes.

JAVASCRIPT

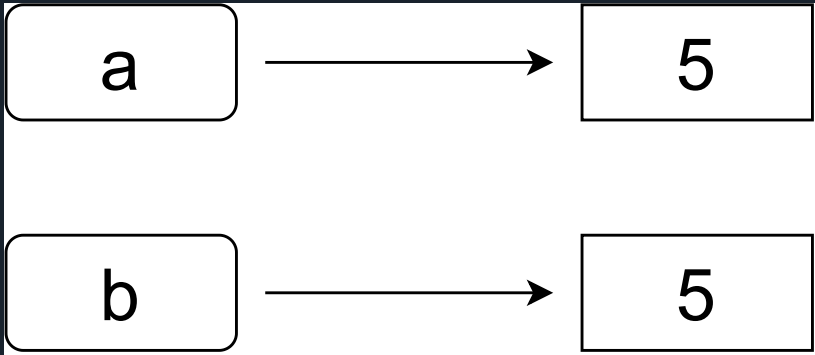
L'immuabilité



No notes on this slide.

JAVASCRIPT

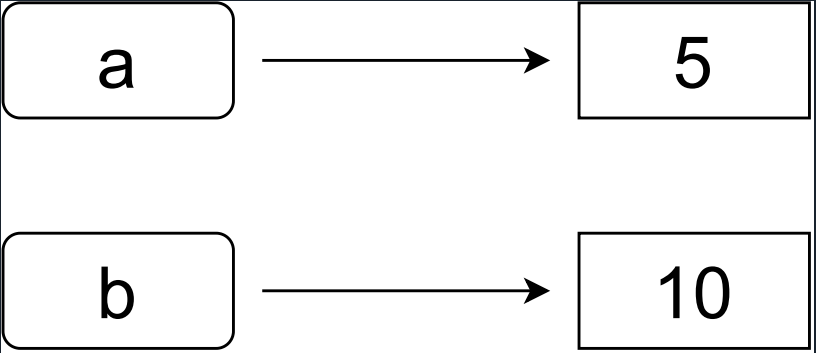
L'immuabilité



No notes on this slide.

JAVASCRIPT

L'immuabilité



En JavaScript les types de base sont *immuables*. Un assignment créera toujours une copie.

Les types de base sont des variables qui **ne changent jamais de valeur**. On dit qu'elles sont immuables. Lors d'un assignment, JavaScript va copier la valeur dans un nouvel espace mémoire, indépendant du premier. Ainsi, modifier la variable `b` dans l'exemple précédent ne modifie pas `a`, car il s'agit de deux variables séparées. En réalité, si vous assignez une nouvelle valeur à `a`, par exemple avec le code :

```
let a = 5;
a = 10;
```

La première ligne assignera à `a` une valeur 5 stockée en mémoire, puis la seconde ligne créera un autre espace mémoire et assignera ce dernier à `a`. La valeur d'origine `5` ne sera pas modifiée. On peut le constater si `a` est une chaîne de caractères :

```
let a = "Bonjour";
a[0] = "A";
```

Faire un `console.log(a)` affichera "Bonjour" et non "Aonjour", même si on a assigné "A" au premier caractère. Cet assignment n'a eu aucun effet car `a` est d'un type de base et donc immuable.

JAVASCRIPT

L'immuabilité

En JavaScript, il existe une différence fondamentale entre les types de base et les objets.

Qu'affiche ce code ?

```
let a = [1, 2, 3];  
let b = a;  
  
b[1] = 10;  
  
console.log(a);  
console.log(b);
```

Ce code affichera :

[1, 10, 3]

[1, 10, 3]

Ici on remarque que modifier `b` modifie également `a`. Le comportement est différent que celui du cas précédent, pourquoi?

JAVASCRIPT

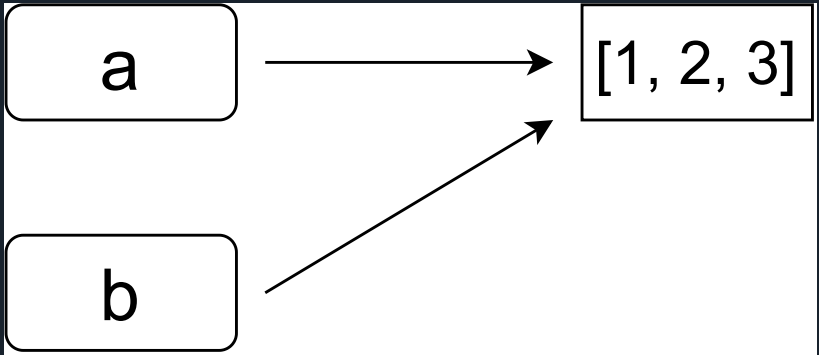
L'immuabilité



No notes on this slide.

JAVASCRIPT

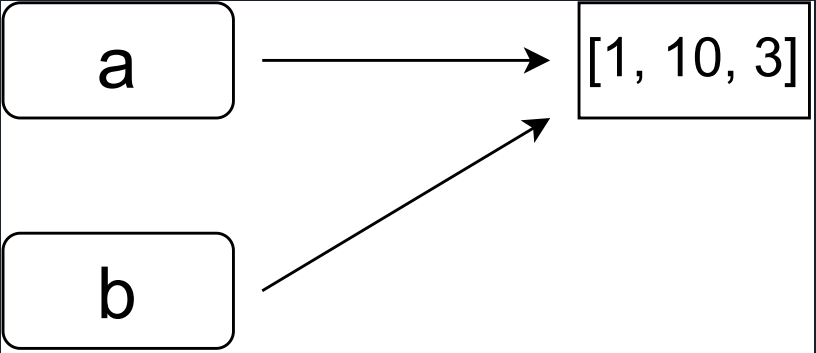
L'immuabilité



No notes on this slide.

JAVASCRIPT

L'immuabilité



En JavaScript les types non basiques sont *mutables*. Un assignment créera une référence vers l'objet.

Contrairement aux types de base, quand on assigne un objet, celui-ci n'est pas copié. La variable assignée pointera vers le même espace mémoire que la variable originale. Ainsi, changer l'une changera implicitement l'autre aussi. Les objets sont **mutable**, il est possible de les changer directement sans créer de copie. C'est bien le comportement attendu :

```
let a = [1,2,3];  
a[0] = 4;
```

Dans ce cas un `console.log(a)` affichera bien `[4,2,3]`, car le tableau a été modifié. Il devient clair pourquoi un code comme celui-ci est valide :

```
const a = [1,2,3];  
a[0] = 10;
```

Bien que `a` soit `const`, il est autorisé de modifier un élément du tableau, car `a` est toujours assigné au même objet. C'est l'objet lui-même que l'on change avec la seconde instruction. Le mot clé `const` ne garanti donc **pas** que l'objet assigné à une variable ne changera pas, il garanti que la variable ne pourra pas être assignée à autre chose.

JAVASCRIPT

Les fonctions

No notes on this slide.

JAVASCRIPT

Les fonctions

Les fonctions se déclarent de la manière suivante

```
function add(a,b,c){    // Déclare la fonction "add", disponible dans le contexte courant
    return a + b + c;
}
const result = add(4,6,8); // result = 18
console.log(typeof add); // Affiche "function"
```

No notes on this slide.

JAVASCRIPT

Les fonctions

En Javascript les fonctions sont des objets comme les autres, il est possible de les assigner à des variables :

```
// Déclare une fonction "foo" assignée à la variable "maFonction"
const maFonction = function foo(a, b) {
  return a * b;
}
console.log(maFonction(3,4)) // Affiche 12
console.log(foo(3,4)) // ERREUR : foo n'est pas défini
```

Les fonctions en JavaScript se déclarent avec une syntaxe similaire au C/C++. Dans le premier exemple on déclare une fonction "add" qui prends 3 paramètres. Le corps de la fonction retourne l'addition des trois paramètres. Comme dans beaucoup de langages, l'instruction "return" retourne la valeur indiquée juste après et met fin à l'exécution de la fonction. En JavaScript les fonctions sont considérées comme des objets, qui peuvent donc être assignés à des variables. Il est possible de créer des tableaux de fonctions, des objets contenant des fonctions, etc. Quand une fonction est assignée à une variable, elle n'est pas disponible dans le contexte actuel. Elle est simplement "stockée" dans la variable

JAVASCRIPT

Les fonctions

Stocker des fonctions dans des variables est très courant en JavaScript. Il existe une syntaxe alternative sans donner de nom à une fonction. Ces fonctions sont dites "anonymes".

```
const operation1 = function nom_inutile(a,b){ return 1 + a * b } // Syntaxe classique
const operation2 = function (a,b){ return 1 + a * b } // Fonction anonyme
```

Les fonctions anonymes sont très utilisées en JavaScript, il existe une notation alternative, plus légère. On les appelle "fonctions fléchées" (*arrow functions*)

```
const operation3 = (a,b) => { return 1 + a * b;} // Syntaxe encore plus simple
```

Si la fonction n'est composée que d'un `return` (comme c'est le cas ici), il est même possible de l'enlever ainsi que les accolades.

```
const operation4 = (a,b) => 1 + a * b; // Dans ce cas, on peut omettre les accolades et le r
```

Toutes les notations de cette slide sont équivalentes, excepté certaines nuances qui sont hors du cadre de ce cours.

On le verra plus loin, mais les fonctions et leur gestion est au coeur du JavaScript. Il est extrêmement commun d'avoir des fonctions dans des objets, ou en tant que paramètres d'autres fonctions, etc.

JAVASCRIPT

Les fonctions

Exemple où une fonction est passée en paramètre d'une fonction

```
function apply_to_table(table, func){ // Le paramètre func est une fonction

    for(let i=0; i < table.length; i++){ // On itère pour chaque élément du tableau
        table[i] = func(table[i]);      // Appel de la fonction en paramètre
    }

    return table;
}

const numbers = [2, 4, 6];
const numbers_applied = apply_to_table(numbers, (x) => 3*x+1);
console.log(numbers_applied); // Affiche [7, 13, 19]
```

Cet exemple illustre la puissance des fonctions en JavaScript : elles peuvent être passées en tant que valeur n'importe où. Du point de vue de JavaScript, les fonctions sont des objets, qui ont la particularité de contenir du code pouvant être exécuté.

JAVASCRIPT

Les fonctions

Les fonctions acceptent des paramètres par défaut

```
function add(a, b, c=0, d=0){  
    return a + b + c + d;  
}  
  
console.log(add(2,3));           // Affiche 5  
console.log(add(2,3,4));        // Affiche 9  
console.log(add(2,3,4,5));      // Affiche 14
```

Les paramètres par défaut doivent toujours se trouver en dernière position

```
function add(a=0, b){}
```

est invalide

Cet exemple illustre la puissance des fonctions en JavaScript : elles peuvent être passées en tant que valeur n'importe où. Du point de vue de JavaScript, les fonctions sont des objets, qui ont la particularité de contenir du code pouvant être exécuté.

JAVASCRIPT

Passage par valeur ou passage par référence

En JavaScript, les types de base (immuables) sont toujours passés **par valeur** aux fonctions.

Leur valeur est copiée lors de l'appel à la fonction.

Modifier un tel paramètre dans la fonction n'aura pas d'effet en dehors.

```
function modify_number(num){ // Type de base en paramètre, il sera dupliqué durant l'exécution
  num = 10;                  // Cette assignation modifie la copie locale dans la fonction
  return num;
}
```

```
const num_1 = 1;
const num_2 = modify_number(num_1);
console.log(num_1);      // Affiche 1 : La valeur de base n'a pas été modifiée
console.log(num_2);      // Affiche 10 : Il s'agit de la valeur renvoyée par la fonction
```

JAVASCRIPT

Passage par valeur ou passage par référence

Tous les autres types (tableaux, objets, fonctions, ets) sont passés **par référence**.

L'élément passé en paramètre n'est pas copié. Une référence vers cet élément est passé à la fonction.

Modifier un tel paramètre dans la fonction aura un effet en dehors de la fonction. On appelle cela un **un effet de bord**.

```
function modify_table(table){ // Objet en paramètre, ce sera une référence
    table[0] = 10;             // Cette assignation va modifier le tableau original
    return table;
}

const table_1 = [1,2,3];
const table_2 = modify_table(table_1);
console.log(table_1); // Retourne [10, 2, 3] : table_1 a été modifié par la fonction
console.log(table_2); // Retourne [10, 2, 3] : il s'agit du même tableau que table_1

// A cet endroit, table_1 et table_2 référencent le même tableau.
// Modifier l'un modifiera aussi l'autre
```

Quand une fonction est appelée, au moment d'assigner la valeur à ses paramètres, elle se comporte comme l'opérateur `=`. C'est-à-dire :

- Les types de base sont immuable, et seront donc dupliqués
- Les objets (types composés) sont mutables, et la fonction s'exécutera *sur le même objet que passé en paramètre*

La fonction **modify_number** prend un nombre en paramètre, et lui assigne 10. Comme il s'agit d'un nombre, il sera copié au moment d'exécuter la fonction, cela ne modifiera pas le nombre d'origine, et la fonction retournera toujours 10. La fonction **modify_table**, au contraire, prend un tableau en paramètre. Le tableau sur lequel va agir la fonction sera le tableau passé en paramètre : il n'y a pas de copie. La ligne `table[1] = 10` va donc modifier le tableau d'origine. De plus, la ligne `return table` va renvoyer le même tableau que passé en paramètre. Les variables `table_1` et `table_2` vont donc référencer le même tableau, et tout changement de l'une sera visible dans l'autre.

Les fonctions qui modifient leur objets d'entrée sans créer de copie sont souvent appelée **in-place**.

JAVASCRIPT

Le DOM

Comment lire et écrire le contenu de notre document HTML avec du JavaScript ? Grâce au Document Object Model (DOM) !

Le DOM est l'interface permettant d'accéder à la page web. Il se caractérise par l'ajout de deux objets, accessibles partout dans le code : `document` et `window`.

```
// Retourne l'élément qui porte l'id "monId"
const e1 = document.getElementById('monId');
// Retourne un tableau avec les éléments de classe "maClasse"
const e2 = document.getElementsByClassName('maClasse');
// Retourne le premier élément correspondant au sélecteur CSS
const e3 = document.querySelector('p .large');
```

Jusqu'ici, nous n'avons utilisé que la console pour interagir avec JavaScript. Évidemment cela n'est pas très pratique et la raison principale d'exécuter du JavaScript dans un navigateur est de pouvoir interagir avec la page web et son contenu.

Le DOM est l'interface que met à disposition le navigateur pour lire / écrire du contenu sur la page.

L'objet `document` offre des fonctions pour interagir avec la page web elle-même et son contenu.

L'objet `window` offre des fonctions pour interagir avec la fenêtre du navigateur. Par exemple, récupérer la résolution de la fenêtre.

JAVASCRIPT

Le DOM

Une fois un élément HTML récupéré, toutes ses propriétés sont modifiables

```
// Sélectionne le premier paragraphe de la page
const elem = document.querySelector('p');
// Assigne un nouveau texte au paragraphe
elem.textContent = "Nouveau contenu du paragraphe";
// Assigne une nouvelle taille de police
elem.style.fontSize = "20pt";
```

Référence du DOM et ses fonctions : https://www.w3schools.com/js/js_htmlDOM.asp

JAVASCRIPT

Le DOM

Il est aussi possible d'attacher des événements aux éléments

```
// Récupère l'élément à modifier
const elem = document.getElementById('MyId');
// Au clic, modifie le texte de l'élément à "Clic!"
elem.addEventListener("click", () => elem.textContent="Clic!");
```

```
// document.body retourne toujours la balise body de la page actuelle
// Quand la page est chargée ("load"), change le texte de l'élément info en "Loaded!"
document.body.addEventListener(
  "load",
  () => document.getElementById('info').textContent="Loaded!"
);
```

La fonction `addEventListener` permet d'attacher une action à un événement.

Le premier paramètre est une chaîne de caractère qui décrit l'événement.

Le second paramètre est l'action à exécuter quand l'événement survient.

Cette action est une fonction, c'est un excellent exemple d'une fonction qui prend en paramètre une autre fonction à exécuter plus tard.

JAVASCRIPT

Le DOM - Les événements

Il est aussi possible de spécifier un événement directement dans la balise

```
<!DOCTYPE html>
<html>
  <head>...</head>
  <body onload="execute_on_load()">
    <button onclick="execute_on_click()">Button</button>
    <script src="script.js"></script>
  </body>
</html>
```

```
function execute_on_load(){console.log("Loaded !");}
function execute_on_click(){console.log("Clicked !");}
```

JAVASCRIPT

Programmation orientée objet

No notes on this slide.

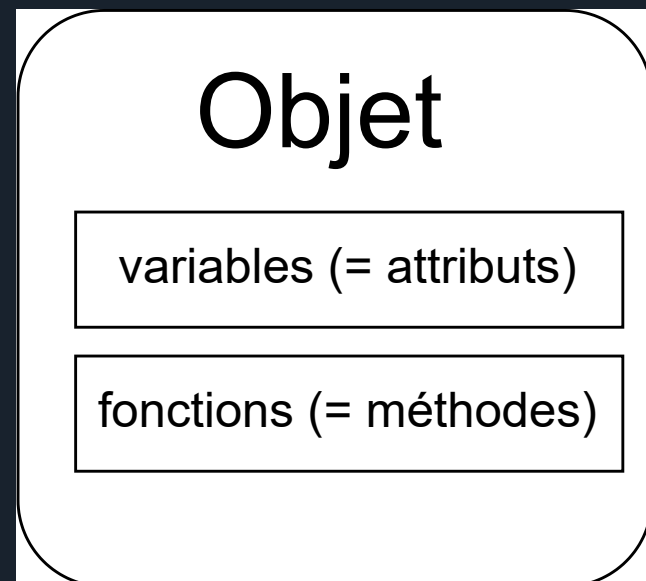
JAVASCRIPT

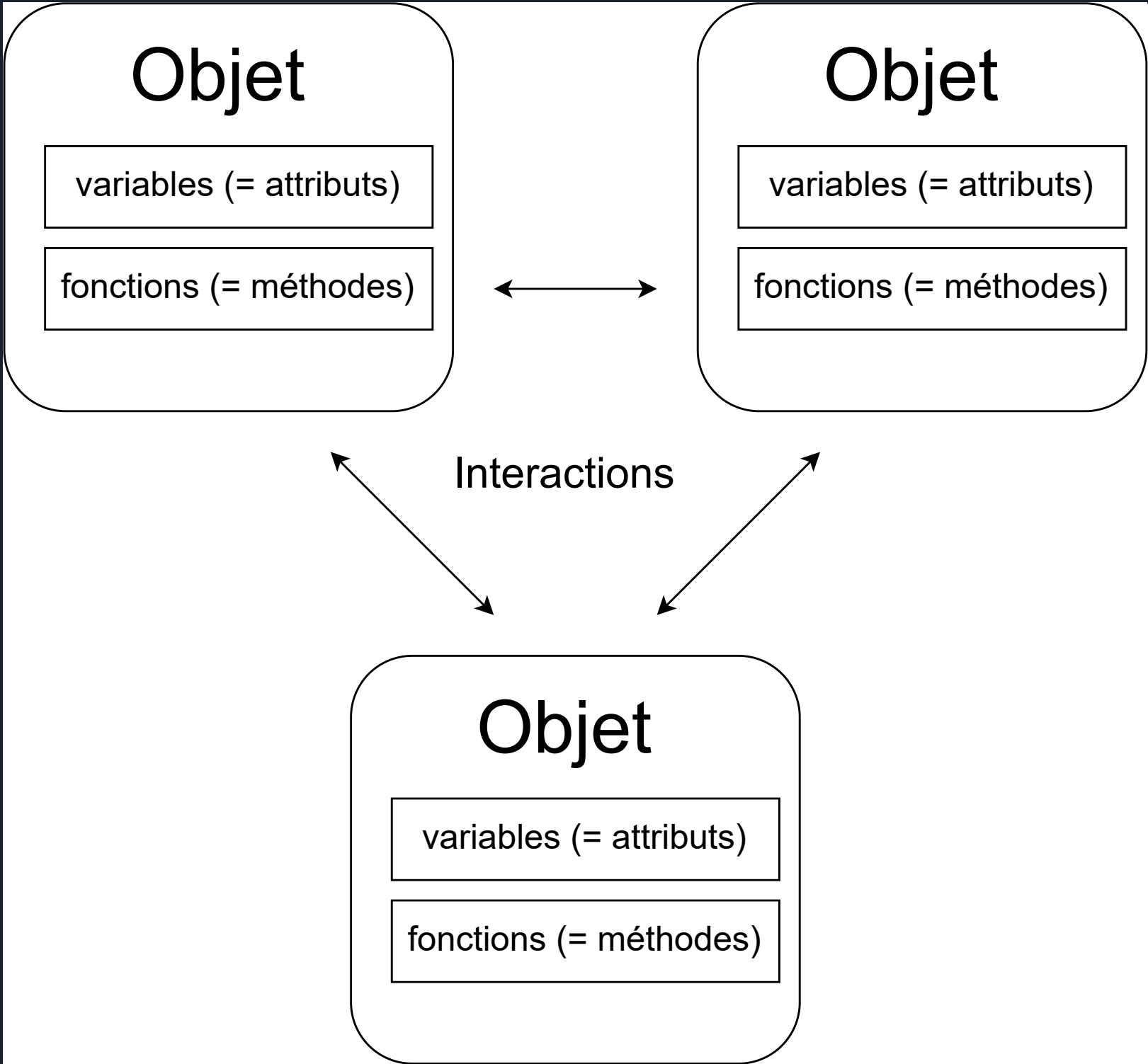
Programmation orientée objet

La programmation orientée objet est une manière de concevoir du code (appelé paradigme) dont le principe est de découper son code en objets. Un objet peut représenter un élément visible à l'écran, un concept concret ou abstrait, ou n'importe quelle "unité" qui possède une logique interne.

Un objet contient deux catégories d'éléments :

- Ses variables internes (nommées attributs), qui définissent son état
- Ses fonctions internes (nommées méthodes), qui définissent ses actions





JAVASCRIPT

Programmation orientée objet

Exemple

```
const rectangle = {  
  height : 300,           // Attribut "height"  
  width : 200,            // Attribut "width"  
  area : function () { return this.height * this.width } // Méthode "area"  
}
```

L'opérateur `this` permet d'obtenir l'objet actuel

```
rectangle.width           // Retourne 200  
rectangle.area()          // Retourne 60000  
rectangle.width = 5  
rectangle.area()          // Retourne 1500
```

Pratique pour des objets simples, mais si on voulait créer plusieurs rectangles il nous faudrait une fonction qui génère des rectangles. Cela deviendrait vite complexe.

On pourrait définir un objet avec le code suivant. Mais cela est peu pratique. - Il faudrait déclarer chaque objet séparément, ou alors créer une fonction spécifique pour créer des rectangles - Pour des objets plus complexes, la syntaxe devient rapidement lourde et difficile à lire

JAVASCRIPT

Programmation orientée objet

Il existe une syntaxe spécialement conçue pour les objets. Une classe est la "description" d'un objet, permettant facilement d'instancier plusieurs objets par la suite.

```
class Rectangle{
  constructor(width, height){ // Constructeur, appelé à la création d'un nouvel objet
    this.width = width;
    this.height = height;
  }
  area(){
    return this.width * this.height;
  }
}
```

Avec le mot clé `new` il est possible d'instancier des classes facilement

```
const rec1 = new Rectangle(20, 30);
const rec2 = new Rectangle(200, 300);
rec1.area(); // 600
rec2.area(); // 60000
```

En utilisant les classes en javascript, il est possible d'avoir une syntaxe facile à lire et permettant d'instancier facilement des objets. La syntaxe utilise le mot clé "class" suivi du nom de la classe. Une classe est une description des propriétés d'un objet, une fois déclarée, une classe ne crée aucun objet. Il faudra utiliser le mot clé "new" pour instancier la classe en objet.

En résumé, une classe est la description d'un objet. Un objet est une instance d'une classe.

Une classe contient une méthode spéciale "constructor" qui s'exécute lors de la création de l'objet. C'est elle qui reçoit les paramètres passés lors de l'instruction "new". Les autres méthodes se déclarent sans le mot clé "function", les unes après les autres dans la classe. Toutes les méthodes ont accès au "this" de l'objet courant.

JAVASCRIPT

Les modules

Il est bien de découper son code en plusieurs fichiers. Par exemple, on pourra définir la classe `Disk` dans son propre fichier, avec également une constante globale pour le cercle unitaire en (0;0).

```
// Fichier Disk.js
class Disk{
  constructor(x, y, radius){
    this.x = x;
    this.y = y;
    this.radius = radius;
  }
  area(){
    return Math.PI*this.radius*this.radius;
  }
}

const UNIT_DISK = new Disk(0, 0, 1);
```

Comment utiliser ce code dans un autre fichier ?

JAVASCRIPT

Les modules

Il serait possible d'importer tous les fichiers dans le HTML principal, à l'aide de balises `<script>`, mais cela a plusieurs inconvénients, en particulier

1. Tout est importé globalement, généralement des conflits de nom. Ainsi, dans l'exemple précédent, il ne peut y avoir un autre objet ou variable se nommant `Disk`.
2. Si une librairie est composée de plusieurs fichiers, ils doivent tous être importés un par un. Cela devient encombrant avec des centaines de fichiers.
3. La gestion des dépendances est très manuelle, complexe, et sujette aux erreurs.

Il existe une solution à tout ces problèmes : **Les modules**

Les modules fournissent aussi, comme on va le voir plus loin, une notion d'encapsulation.

JAVASCRIPT

Les modules

Un module est un code contenu dans un fichier qui s'exécute dans un contexte isolé. Tout ce qui n'est pas exporté d'un module ne sera pas visible depuis l'extérieur.

Pour exporter un élément, on utilise le mot clé export.

```
// Fichier Disk.js
export class Disk{                                // Classe exportée
  constructor(x, y, radius){
    this.x = x;
    this.y = y;
    this.radius = radius;
  }
  area(){
    return Math.PI*this.radius*this.radius;
  }
}

export const UNIT_DISK_AREA = new Disk(0, 0, 1).area(); // Constante exportée
const UNIT_DISK_PERIMETER = 2*Math.PI;                 // Constante non exportée
```

JAVASCRIPT

Les modules

Pour importer un élément d'un module, on utilisera le mot-clé `import`

```
// Fichier main.js
import { Disk, UNIT_DISK_AREA } from "./Disk.js"

const myDisk = new Disk(2, 3, 1.5);

if(myDisk.area() > UNIT_DISK_AREA) {
    console.log("myDisk plus grand que le cercle unité");
}

import { UNIT_DISK_PERIMETER } from "./Disk.js" // Erreur ! Car non exporté
```

Les imports seront indiqués entre `{` et `}`, même si un seul élément est importé

On ne peut importer des éléments que dans un module. En utilisant ce principe tout notre code sera un ensemble de modules.

Il est courant de faire du re-export, c'est-à-dire d'importer des éléments dans un module et de les y exporter. Cela permet à un utilisateur de ne pas faire trop d'imports différents. Par exemple, si vous créez une librairie avec un module "Button.js" et un module "Slider.js", vous pouvez créer un module "Widget.js" qui importe puis réexporte Button et Slider. En utilisant votre librairie il suffira de faire "import {Button, Slider} from 'Widget.js'

JAVASCRIPT

Les modules

Pour chaque module, il est possible d'indiquer un export par défaut

```
// Fichier Disk.js
export default class Disk{ ... } // Export par défaut
export const UNIT_DISK = new Disk(0, 0, 1);
```

L'export par défaut peut être importé en lui donnant n'importe quel nom, sans accolades. JavaScript sait qu'un import sans accolades est forcément l'export par défaut du module.

```
// Fichier main.js
import Disk from "./Disk.js" // Valide
const disk = new Disk(1,2,3);
```

```
// Fichier main.js
import Cercle from "./Disk.js" // Également valide, ici Disk est appelé Cercle
const disk = new Cercle(1,2,3);
```

JAVASCRIPT

Les modules

Il est possible de renommer un import avec le mot-clé `as`. Cela est particulièrement utile quand on importe deux éléments dont les noms sont les mêmes.

```
import { Disk as Cercle, UNIT_DISK as CERCLE_UNITE } from "./Disk.js"
```

C'est typiquement le cas quand on utilise plusieurs librairies différentes dans un seul fichier

```
import { Color as ColorLib1 } from "./my/library1/Color.js"
import { Color as ColorLib2 } from "./my/library2/Color.js"
```

JAVASCRIPT

Les modules

Il est possible de mélanger les imports et les imports par défaut

```
// Importe l'import pas défaut sous le nom de "Cercle",  
// et l'import "UNIT_DISK" sous le nom de "CERCLE_UNITE"  
import Cercle, {UNIT_DISK as CERCLE_UNITE} from "../Disk.js"
```

Attention à ne pas abuser du renommage. A utiliser seulement pour éviter des collisions de noms ou éviter des ambiguïtés, sinon la compréhension du code devient très difficile.

JAVASCRIPT

Les modules

Pour importer un module JavaScript dans du HTML, on utilise la balise `<script>` avec `type="module"`

```
<script type="module">
  import Disk from "./Disk.js";
  /* ... */
</script>
```

Attention : l'isolation vaut toujours dans ce cas, et le contenu de la balise `script` sera isolé du reste du code

```
<script>
  const value = 10;
</script>
<script type="module">
  console.log(value); // value n'existe pas ici, affichera undefined
</script>
<script>
  console.log(value); // affichera 10
</script>
```